TRD

Technical Requirements Document (TRD)

AWS Glue ETL Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Technical Requirements Document translates the functional requirements into implementation-ready technical specifications for an AWS Glue-based ETL pipeline. It defines the technical architecture, data structures, transformation logic, and operational parameters required to build a production-grade solution that ingests customer and order data, performs data quality operations, implements SCD Type 2 using Apache Hudi, and produces analytical aggregates.

Scope

This TRD covers:

- AWS Glue Spark job technical specifications
- Source-to-target data mappings with schema definitions
- Apache Hudi configuration for SCD Type 2 implementation
- AWS Glue Data Catalog integration patterns
- Data quality validation and error handling logic
- Incremental processing technical approach
- Aggregation computation specifications
- S3 storage patterns and file formats
- Runtime parameter definitions

This TRD does NOT cover:

- AWS infrastructure provisioning (VPC, subnets, security groups)
- IAM role and policy definitions
- AWS Glue job scheduling or orchestration configuration
- Cost optimization strategies
- Disaster recovery procedures
- Performance tuning beyond standard Glue configurations

Assumptions

1. AWS Glue version 3.0 or higher will be used (supports Spark 3.1+ and Hudi 0.10+)
2. CSV files use comma delimiter with header row

3. CSV encoding is UTF-8

4. Date fields in Order dataset follow ISO 8601 format (YYYY-MM-DD) or can be parsed by Spark's default date parser

5. Numeric fields (PricePerUnit, Qty) are stored as strings in CSV and require casting to decimal/integer types

6. CustId is a string type serving as the business key for joins and SCD2 tracking

7. OpTs (Operation Timestamp) will be generated using current_timestamp() function at processing time

8. Hudi table type will be COPY_ON_WRITE for ordersummary table

9. Hudi record key for ordersummary will be a composite of OrderId and StartDate

10. Hudi precombine field will be OpTs

11. Glue Data Catalog database "gen_ai_poc_databrickscoe" is pre-created

12. S3 bucket "adif-sdlc" exists with appropriate bucket policies

13. Glue job will run with sufficient DPU allocation (minimum 2 DPUs recommended)

14. No schema evolution is expected during initial implementation

15. customeraggregatespend table will be fully refreshed on each run (not incremental)

16. "Null" string matching is case-sensitive

17. Duplicate detection considers all columns with exact match logic

18. Customer changes are detected by comparing full dataset snapshots (no CDC mechanism)

19. All S3 writes will use Parquet format except where Hudi format is specified

20. Glue job will use dynamic frame to catalog table conversions for metadata management

•

2. Technical Requirements

TR-INGEST-001: Implement Customer Data Ingestion

• Related Functional Requirement ID(s): FR-INGEST-001

• Objective:

Read customer CSV files from S3 source location into AWS Glue DynamicFrame for processing, applying schema inference and validation.

• Source Details:

• Source Type: S3 CSV Files

• Source Path: s3://adif-sdlc/sdlc_wizard/customerdata/

• Source Format: CSV

• Source Delimiter: Comma (,)

• Source Header: Present (first row)

• Source Encoding: UTF-8

• Target Details:

• Target Type: In-memory Glue DynamicFrame

• Target Name: customer_raw_df

• Input Schema:

Not enough information available in the FRD to derive complete schema details for customer source. Based on FRD column names only:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	(not specified)	(not specified)	(not specified)	
	Name	(not specified)	(not specified)	(not specified)	
	EmailId	(not specified)	(not specified)	(not specified)	
	Region	(not specified)	(not specified)	(not specified)	

• Output Schema:

Same as Input Schema (pass-through at this stage)

• Processing / Transformation Logic:

1. Initialize GlueContext and SparkSession

2. Use glueContext.create_dynamic_frame.from_options() with format="csv"

3. Configure connection options:

• paths: ["s3://adif-sdlc/sdlc_wizard/customerdata/"]

• format: "csv"

• format_options: {"withHeader": True, "separator": ","}

4. Apply schema inference from CSV header row

5. Load all CSV files matching the path pattern into single DynamicFrame

6. Validate that expected columns (CustId, Name, EmailId, Region) are present

• Validation Rules:

• Source path must exist and be accessible

• At least one CSV file must be present in source path

• CSV files must contain header row with expected column names

• CSV files must be parseable with comma delimiter

• All expected columns must be present in header

• Error Handling and Logging:

• Log INFO: Start of customer data ingestion with source path

- Log INFO: Number of files discovered in source path
- Log ERROR and FAIL job if source path does not exist
- Log ERROR and FAIL job if no CSV files found in source path
- Log ERROR and FAIL job if CSV parsing fails with file name and error details
- Log ERROR and FAIL job if expected columns are missing with schema mismatch details
- Log INFO: Successful ingestion with record count

- Runtime Parameters:
- customer_source_path: s3://adif-sdlc/sdlc_wizard/customerdata/ (default, overridable)

- Operational Notes:
- Use Glue DynamicFrame for schema flexibility and catalog integration
- Leverage Glue's built-in CSV reader for optimized performance
- Ensure Glue job has s3:GetObject permission on source bucket
-

TR-INGEST-002: Implement Order Data Ingestion

- Related Functional Requirement ID(s): FR-INGEST-002

- Objective:

Read order CSV files from S3 source location into AWS Glue DynamicFrame for processing, applying schema inference and validation.

- Source Details:
- Source Type: S3 CSV Files
- Source Path: s3://adif-sdlc/sdlc_wizard/orderdata/
- Source Format: CSV
- Source Delimiter: Comma (,)
- Source Header: Present (first row)
- Source Encoding: UTF-8

- Target Details:
- Target Type: In-memory Glue DynamicFrame
- Target Name: order_raw_df

- Input Schema:

Not enough information available in the FRD to derive complete schema details for order source. Based on FRD column names only:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	----------	----------	----------	----------	
	OrderId	(not specified)	(not specified)	(not specified)	
	ItemName	(not specified)	(not specified)	(not specified)	
	PricePerUnit	(not specified)	(not specified)	(not specified)	
	Qty	(not specified)	(not specified)	(not specified)	
	Date	(not specified)	(not specified)	(not specified)	
	CustId	(not specified)	(not specified)	(not specified)	

• Output Schema:

Same as Input Schema (pass-through at this stage)

• Processing / Transformation Logic:

1. Use glueContext.create_dynamic_frame.from_options() with format="csv"

2. Configure connection options:

• paths: ["s3://adif-sdlc/sdlc_wizard/orderdata/"]

• format: "csv"

• format_options: {"withHeader": True, "separator": ","}

3. Apply schema inference from CSV header row

4. Load all CSV files matching the path pattern into single DynamicFrame

5. Validate that expected columns (OrderId, ItemName, PricePerUnit, Qty, Date, CustId) are present

• Validation Rules:

• Source path must exist and be accessible

• At least one CSV file must be present in source path

• CSV files must contain header row with expected column names

• CSV files must be parseable with comma delimiter

• All expected columns must be present in header

• Error Handling and Logging:

• Log INFO: Start of order data ingestion with source path

• Log INFO: Number of files discovered in source path

• Log ERROR and FAIL job if source path does not exist

• Log ERROR and FAIL job if no CSV files found in source path

• Log ERROR and FAIL job if CSV parsing fails with file name and error details

• Log ERROR and FAIL job if expected columns are missing with schema mismatch details

• Log INFO: Successful ingestion with record count

- Runtime Parameters:
- order_source_path: s3://adif-sdlc/sdlc_wizard/orderdata/ (default, overridable)

- Operational Notes:
- Use Glue DynamicFrame for schema flexibility and catalog integration
- Leverage Glue's built-in CSV reader for optimized performance
- Ensure Glue job has s3:GetObject permission on source bucket

-

TR-DQ-001: Implement Data Quality Processing

- Related Functional Requirement ID(s): FR-DQ-001

- Objective:

Apply data quality rules to remove null values (both string "Null" and actual NULL) and duplicate records from customer and order datasets.

- Source Details:
- Source Type: In-memory Glue DynamicFrame
- Source Name (Customer): customer_raw_df (from TR-INGEST-001)
- Source Name (Order): order_raw_df (from TR-INGEST-002)

- Target Details:
- Target Type: In-memory Glue DynamicFrame
- Target Name (Customer): customer_clean_df
- Target Name (Order): order_clean_df

- Input Schema:
- Customer: Same as TR-INGEST-001 output
- Order: Same as TR-INGEST-002 output

- Output Schema:
- Customer: Same as input schema (subset of records)
- Order: Same as input schema (subset of records)

- Processing / Transformation Logic:

For Customer Dataset:

1. Convert DynamicFrame to Spark DataFrame for transformation operations
2. Apply null removal filter:

• For each column in schema, create filter condition: (col(column_name) != "Null") AND (col(column_name).isNotNull())

• Combine all column conditions with AND operator

• Apply combined filter to DataFrame

3. Apply deduplication:

• Use dropDuplicates() without parameters to consider all columns

4. Convert cleaned DataFrame back to DynamicFrame

5. Store result in customer_clean_df

For Order Dataset:

1. Convert DynamicFrame to Spark DataFrame for transformation operations

2. Apply null removal filter:

• For each column in schema, create filter condition: (col(column_name) != "Null") AND (col(column_name).isNotNull())

• Combine all column conditions with AND operator

• Apply combined filter to DataFrame

3. Apply deduplication:

• Use dropDuplicates() without parameters to consider all columns

4. Convert cleaned DataFrame back to DynamicFrame

5. Store result in order_clean_df

• Validation Rules:

• String "Null" matching is case-sensitive (exact match)

• Duplicate detection considers exact match across all columns

• At least one record must remain after cleaning (warning if zero)

• Error Handling and Logging:

• Log INFO: Start of data quality processing for customer dataset

• Log INFO: Customer records before cleaning: {count}

• Log INFO: Customer records after null removal: {count}

• Log INFO: Customer records after deduplication: {count}

• Log WARNING if all customer records removed: "All customer records removed by data quality rules"

• Log INFO: Start of data quality processing for order dataset

• Log INFO: Order records before cleaning: {count}

• Log INFO: Order records after null removal: {count}

• Log INFO: Order records after deduplication: {count}

• Log WARNING if all order records removed: "All order records removed by data quality rules"
• Continue processing even if all records removed (downstream steps will handle empty datasets)

• Runtime Parameters:
• None (uses in-memory datasets)

• Operational Notes:
• Use Spark DataFrame operations for efficient filtering and deduplication
• Consider caching DataFrames if used multiple times in downstream operations
• Null removal logic must handle both string "Null" and actual NULL values

•

TR-CATALOG-001: Implement Glue Catalog Registration

• Related Functional Requirement ID(s): FR-CATALOG-001

• Objective:

Register cleaned customer and order datasets as tables in AWS Glue Data Catalog for metadata management and query access.

• Source Details:
• Source Type: In-memory Glue DynamicFrame
• Source Name (Customer): customer_clean_df (from TR-DQ-001)
• Source Name (Order): order_clean_df (from TR-DQ-001)

• Target Details:
• Target Type: AWS Glue Data Catalog Table
• Target Database: gen_ai_poc_databrickscoe
• Target Table (Customer): sdlc_wizard_customer
• Target Table (Order): sdlc_wizard_order
• Target Storage Format: Parquet
• Target Storage Location (Customer): s3://adif-sdlc/catalog/sdlc_wizard/customer/
• Target Storage Location (Order): s3://adif-sdlc/catalog/sdlc_wizard/order/

• Input Schema:
• Customer: Same as TR-DQ-001 output
• Order: Same as TR-DQ-001 output

• Output Schema:
• Customer: Same as input schema (persisted in catalog)

• Order: Same as input schema (persisted in catalog)

• Processing / Transformation Logic:

For Customer Table:

1. Use glueContext.write_dynamic_frame.from_catalog() method
2. Configure write parameters:
• database: "gen_ai_poc_databrickscoe"
• table_name: "sdlc_wizard_customer"
• transformation_ctx: "customer_catalog_write"
3. If table does not exist, create new table with inferred schema
4. If table exists, overwrite data and update metadata
5. Write data in Parquet format to S3 location
6. Update catalog metadata with current timestamp

For Order Table:

1. Use glueContext.write_dynamic_frame.from_catalog() method
2. Configure write parameters:
• database: "gen_ai_poc_databrickscoe"
• table_name: "sdlc_wizard_order"
• transformation_ctx: "order_catalog_write"
3. If table does not exist, create new table with inferred schema
4. If table exists, overwrite data and update metadata
5. Write data in Parquet format to S3 location
6. Update catalog metadata with current timestamp

• Validation Rules:
• Target database must exist in Glue Data Catalog
• Table schemas must match source DynamicFrame schemas
• S3 target locations must be writable

• Error Handling and Logging:
• Log INFO: Start of catalog registration for customer table
• Log INFO: Customer records written to catalog: {count}
• Log ERROR and FAIL job if database does not exist
• Log ERROR and FAIL job if table creation fails with permission details
• Log ERROR and FAIL job if S3 write fails with path and error details
• Log INFO: Customer table registered successfully in catalog

• Log INFO: Start of catalog registration for order table
• Log INFO: Order records written to catalog: {count}
• Log ERROR and FAIL job if table creation fails with permission details
• Log ERROR and FAIL job if S3 write fails with path and error details
• Log INFO: Order table registered successfully in catalog

• Runtime Parameters:
• catalog_database: gen_ai_poc_databrickscoe (default, overridable)
• customer_table_name: sdlc_wizard_customer (default, overridable)
• order_table_name: sdlc_wizard_order (default, overridable)

• Operational Notes:
• Use Glue's native catalog integration for automatic metadata management
• Parquet format provides efficient storage and query performance
• Tables will be queryable via Amazon Athena after registration
• Ensure Glue job has glue:CreateTable and glue:UpdateTable permissions
• Ensure Glue job has s3:PutObject permission on target locations

•

TR-SCD2-001: Implement SCD Type 2 for Order Summary

• Related Functional Requirement ID(s): FR-SCD2-001

• Objective:

Join customer and order data, then implement Slowly Changing Dimension Type 2 logic using Apache Hudi to maintain historical tracking of order summaries with customer attribute changes.

• Source Details:
• Source Type (Customer): AWS Glue Data Catalog Table
• Source Database: gen_ai_poc_databrickscoe
• Source Table (Customer): sdlc_wizard_customer
• Source Type (Order): AWS Glue Data Catalog Table
• Source Database: gen_ai_poc_databrickscoe
• Source Table (Order): sdlc_wizard_order
• Source Type (Existing Hudi): S3 Hudi Table (if exists)
• Source Path (Existing Hudi): s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Target Details:
• Target Type: Apache Hudi Table (COPY_ON_WRITE)

• Target Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Target Database: gen_ai_poc_databrickscoe

• Target Table: ordersummary

• Hudi Table Type: COPY_ON_WRITE

• Hudi Operation: UPSERT

• Input Schema:

Customer (from sdlc_wizard_customer):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	(not specified)	(not specified)	(not specified)	
	Name	(not specified)	(not specified)	(not specified)	
	EmailId	(not specified)	(not specified)	(not specified)	
	Region	(not specified)	(not specified)	(not specified)	

Order (from sdlc_wizard_order):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	(not specified)	(not specified)	(not specified)	
	ItemName	(not specified)	(not specified)	(not specified)	
	PricePerUnit	(not specified)	(not specified)	(not specified)	
	Qty	(not specified)	(not specified)	(not specified)	
	Date	(not specified)	(not specified)	(not specified)	
	CustId	(not specified)	(not specified)	(not specified)	

• Output Schema:

ordersummary (Hudi table):

Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping
---	---	---	---	---
OrderId	(not specified)	(not specified)	(not specified)	Direct map from Order.OrderId
ItemName	(not specified)	(not specified)	(not specified)	Direct map from Order.ItemName
PricePerUnit	(not specified)	(not specified)	(not specified)	Direct map from Order.PricePerUnit
Qty	(not specified)	(not specified)	(not specified)	Direct map from Order.Qty
Date	(not specified)	(not specified)	(not specified)	Direct map from Order.Date
CustId	(not specified)	(not specified)	(not specified)	Direct map from Customer.CustId / Order.CustId
Name	(not specified)	(not specified)	(not specified)	Direct map from Customer.Name
EmailId	(not specified)	(not specified)	(not specified)	Direct map from Customer.EmailId
Region	(not specified)	(not specified)	(not specified)	Direct map from Customer.Region

IsActive	Boolean	N/A	No	Derived: True for active records, False for closed records
StartDate	Date	N/A	No	Derived: current_date() for new/updated records
EndDate	Date	N/A	Yes	Derived: NULL for active records, current_date() for closed records
OpTs	Timestamp	N/A	No	Derived: current_timestamp() at processing time

• Processing / Transformation Logic:

1. Read Source Data:

• Read customer data from catalog table: gen_ai_poc_databrickscoe.sdlc_wizard_customer

• Read order data from catalog table: gen_ai_poc_databrickscoe.sdlc_wizard_order

• Attempt to read existing Hudi table from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (if exists)

2. Join Customer and Order Data:

• Perform inner join: order_df.join(customer_df, on="CustId", how="inner")

• Result contains all order fields + all customer fields

3. Add SCD2 Tracking Fields to Joined Data:

• Add IsActive column: lit(True)

• Add StartDate column: current_date()

• Add EndDate column: lit(None).cast("date")

• Add OpTs column: current_timestamp()

4. Implement SCD2 Logic:

If existing Hudi table does NOT exist (initial load):

• Write joined data with SCD2 fields directly to Hudi table

• All records will have IsActive=True, EndDate=NULL

If existing Hudi table EXISTS (incremental update):

• Read existing active records: filter where IsActive = True

• Define business key for comparison: OrderId (primary identifier for order)

• Define change detection attributes: Name, EmailId, Region (customer attributes)

• For each record in joined data:

• Check if OrderId exists in existing active records

• If NOT exists: Insert as new record with IsActive=True, StartDate=current_date(), EndDate=NULL

• If EXISTS:

• Compare change detection attributes (Name, EmailId, Region)

• If NO CHANGE: Skip (no action needed)

• If CHANGE DETECTED:

• Mark existing record as inactive: IsActive=False, EndDate=current_date()

• Insert new record with updated attributes: IsActive=True, StartDate=current_date(), EndDate=NULL

5. Hudi Write Configuration:

• Table Type: COPY_ON_WRITE

• Record Key: OrderId

• Precombine Key: OpTs

• Partition Path: (none - unpartitioned for simplicity)

• Operation: UPSERT

• Hudi Write Options:

• hoodie.table.name: ordersummary

• hoodie.datasource.write.recordkey.field: OrderId

• hoodie.datasource.write.precombine.field: OpTs

• hoodie.datasource.write.operation: upsert

• hoodie.datasource.write.table.type: COPY_ON_WRITE

• hoodie.datasource.hive_sync.enable: true

• hoodie.datasource.hive_sync.database: gen_ai_poc_databrickscoe

• hoodie.datasource.hive_sync.table: ordersummary

• hoodie.datasource.hive_sync.partition_extractor_class: org.apache.hudi.hive.NonPartitionedExtractor

• hoodie.datasource.hive_sync.use_jdbc: false

6. Write to Hudi Table:

• Write DataFrame to s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ using Hudi format

• Hudi will automatically sync metadata to Glue Data Catalog

• Validation Rules:

• CustId must be present in both customer and order datasets for join

• OrderId serves as unique business key for SCD2 tracking

• Change detection compares Name, EmailId, Region attributes

• IsActive must be Boolean type

• StartDate and EndDate must be Date type

• OpTs must be Timestamp type

• At any point in time, each OrderId should have exactly one active record (IsActive=True)

• Error Handling and Logging:

• Log INFO: Start of SCD2 processing for ordersummary

• Log INFO: Customer records read from catalog: {count}

• Log INFO: Order records read from catalog: {count}

• Log INFO: Joined records (customer + order): {count}

• Log WARNING if join produces zero records: "No matching records between customer and order datasets"

• Log INFO: Existing Hudi table found: {True/False}

• Log INFO: Existing active records in Hudi table: {count}

• Log INFO: New records to insert: {count}

• Log INFO: Records with changes detected: {count}

• Log INFO: Records with no changes: {count}

• Log ERROR and FAIL job if CustId missing in either dataset

• Log ERROR and FAIL job if Hudi write fails with error details

• Log ERROR and FAIL job if duplicate OrderIds detected in source data

• Log INFO: SCD2 processing completed successfully

• Log INFO: Total records in ordersummary table: {count}

• Log INFO: Active records in ordersummary table: {count}

• Runtime Parameters:

• ordersummary_target_path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (default, overridable)

• hudi_table_name: ordersummary (default, overridable)

• Operational Notes:

• Hudi COPY_ON_WRITE table type provides snapshot isolation for queries

• Hudi automatic Glue catalog sync eliminates manual catalog registration

• Time-travel queries supported via Hudi's built-in versioning

• Ensure Glue job has Hudi libraries available (Glue 3.0+ includes Hudi)

• Ensure sufficient DPU allocation for Hudi write operations (minimum 2 DPUs)

• Consider partitioning by Date if dataset grows large (not implemented in initial version)

•

TR-INCR-001: Implement Incremental Customer Update Processing

• Related Functional Requirement ID(s): FR-INCR-001

• Objective:

Detect changes in customer master data and trigger corresponding SCD Type 2 updates in the ordersummary Hudi table to maintain historical accuracy.

• Source Details:

• Source Type (New Customer Data): S3 CSV Files

• Source Path: s3://adif-sdlc/sdlc_wizard/customerdata/

• Source Type (Existing Hudi Table): S3 Hudi Table

• Source Path (Hudi): s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Target Details:

• Target Type: Apache Hudi Table (COPY_ON_WRITE)

• Target Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Target Database: gen_ai_poc_databrickscoe

• Target Table: ordersummary

• Input Schema:

New Customer Data (from s3://adif-sdlc/sdlc_wizard/customerdata/):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	(not specified)	(not specified)	(not specified)	
	Name	(not specified)	(not specified)	(not specified)	
	EmailId	(not specified)	(not specified)	(not specified)	
	Region	(not specified)	(not specified)	(not specified)	

Existing ordersummary (from Hudi table):

Same as TR-SCD2-001 output schema

• Output Schema:

Same as TR-SCD2-001 output schema (updated records)

• Processing / Transformation Logic:

1. Read New Customer Data:

• Read customer CSV files from s3://adif-sdlc/sdlc_wizard/customerdata/

• Apply data quality rules (null removal, deduplication) as per TR-DQ-001

• Store in new_customer_df

2. Read Existing ordersummary Hudi Table:

• Read Hudi table from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Filter for active records: where IsActive = True

• Store in existing_ordersummary_df

3. Detect Customer Changes:

• Extract unique customers from existing_ordersummary_df: select distinct CustId, Name, EmailId, Region

• Join with new_customer_df on CustId

• Compare attributes: Name, EmailId, Region

• Identify changed customers: where any attribute differs between old and new

• Store changed customer CustIds in changed_customers_list

4. Process Changes for Each Changed Customer:

For each CustId in changed_customers_list:

a. Close Existing Active Records:

• Filter existing_ordersummary_df for records where CustId matches and IsActive = True

• Update these records:

• Set IsActive = False

• Set EndDate = current_date()

• Keep OpTs unchanged (preserve original timestamp)

b. Create New Active Records:

• Take the same filtered records from step (a)

• Update customer attributes (Name, EmailId, Region) with values from new_customer_df

• Set IsActive = True

• Set StartDate = current_date()

• Set EndDate = NULL

• Set OpTs = current_timestamp()

c. Combine Updates:

• Union closed records and new active records

• This creates the full set of updates for this customer

5. Write Updates to Hudi Table:

• Combine all customer updates into single DataFrame

• Write to Hudi table using UPSERT operation

• Hudi configuration same as TR-SCD2-001

• Hudi will handle merging updates with existing records based on OrderId (record key)

6. Handle Customer Deletions (Optional Logic):

• If a CustId exists in existing_ordersummary_df but NOT in new_customer_df:
• This indicates customer deletion
• Close all active records for that CustId: IsActive=False, EndDate=current_date()
• Note: This logic assumes full refresh of customer data (not explicitly stated in FRD)

• Validation Rules:
• New customer data must pass data quality checks (TR-DQ-001)
• CustId must exist in both old and new customer datasets for change detection
• Change detection compares Name, EmailId, Region attributes
• All SCD2 tracking fields must maintain integrity (IsActive, StartDate, EndDate, OpTs)
• After update, each OrderId should have exactly one active record per customer version

• Error Handling and Logging:
• Log INFO: Start of incremental customer update processing
• Log INFO: New customer records loaded: {count}
• Log INFO: Existing active ordersummary records: {count}
• Log INFO: Unique customers in existing ordersummary: {count}
• Log INFO: Customer changes detected: {count}
• Log INFO if zero changes: "No customer changes detected, skipping SCD2 updates"
• Log INFO: Processing updates for CustId: {CustId}
• Log INFO: Active records to close for CustId {CustId}: {count}
• Log INFO: New active records to create for CustId {CustId}: {count}
• Log INFO: Total records to upsert: {count}
• Log ERROR and FAIL job if Hudi upsert fails with transaction error details
• Log INFO: Incremental update completed successfully
• Log INFO: Updated ordersummary table - Total records: {count}, Active records: {count}

• Runtime Parameters:
• customer_source_path: s3://adif-sdlc/sdlc_wizard/customerdata/ (default, overridable)
• ordersummary_hudi_path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (default, overridable)

• Operational Notes:
• This logic assumes full refresh of customer data (compare entire dataset)
• For true incremental processing, consider implementing CDC (Change Data Capture) mechanism
• Hudi's UPSERT operation ensures atomic updates
• Consider implementing watermark or timestamp-based change detection for large datasets
• This requirement can be integrated into TR-SCD2-001 as a unified SCD2 processing module

• Ensure sufficient DPU allocation for large-scale updates

•

TR-AGG-001: Implement Customer Aggregate Spend Calculation

• Related Functional Requirement ID(s): FR-AGG-001

• Objective:

Aggregate order data by customer to calculate total spending amounts and store results in a dedicated analytics table for reporting purposes.

• Source Details:

• Source Type (Customer): AWS Glue Data Catalog Table

• Source Database: gen_ai_poc_databrickscoe

• Source Table (Customer): sdlc_wizard_customer

• Source Type (Order): AWS Glue Data Catalog Table

• Source Database: gen_ai_poc_databrickscoe

• Source Table (Order): sdlc_wizard_order

• Target Details:

• Target Type: S3 Parquet Files + Glue Catalog Table

• Target Path: s3://adif-sdlc/analytics/customeraggregatespend/

• Target Database: gen_ai_poc_databrickscoe

• Target Table: customeraggregatespend

• Target Format: Parquet

• Write Mode: Overwrite (full refresh)

• Input Schema:

Customer (from sdlc_wizard_customer):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	(not specified)	(not specified)	(not specified)	
	Name	(not specified)	(not specified)	(not specified)	
	EmailId	(not specified)	(not specified)	(not specified)	
	Region	(not specified)	(not specified)	(not specified)	

Order (from sdlc_wizard_order):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	

	OrderId	(not specified)	(not specified)	(not specified)	
	ItemName	(not specified)	(not specified)	(not specified)	
	PricePerUnit	(not specified)	(not specified)	(not specified)	
	Qty	(not specified)	(not specified)	(not specified)	
	Date	(not specified)	(not specified)	(not specified)	
	CustId	(not specified)	(not specified)	(not specified)	

• Output Schema:

customeraggregatespend:

Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping
---	----------	----------	----------	----------
	(not specified)	(not specified)	No	Direct map from Customer.Name
ount	Decimal	(not specified)	No	Derived: SUM(Order.PricePerUnit × Order.Qty) grouped by Customer.Name and Or
	(not specified)	(not specified)	No	Direct map from Order.Date

• Processing / Transformation Logic:

1. Read Source Data:

• Read customer data from catalog: gen_ai_poc_databrickscoe.sdlc_wizard_customer

• Read order data from catalog: gen_ai_poc_databrickscoe.sdlc_wizard_order

2. Join Customer and Order Data:

• Perform inner join: order_df.join(customer_df, on="CustId", how="inner")

• Result contains: OrderId, ItemName, PricePerUnit, Qty, Date, CustId, Name, EmailId, Region

3. Calculate Line-Level Amount:

• Add calculated column: Amount = PricePerUnit × Qty

• Cast PricePerUnit to Decimal type if not already numeric

• Cast Qty to Integer or Decimal type if not already numeric

• Handle non-numeric values: exclude records where casting fails

4. Aggregate by Customer and Date:

• Group by: Name, Date

• Aggregate: TotalAmount = SUM(Amount)

• Result schema: Name, Date, TotalAmount

5. Select Final Columns:

• Select: Name, TotalAmount, Date

• Order by: Name, Date (optional, for consistent output)

6. Write to Target:

• Write DataFrame to S3 path: s3://adif-sdlc/analytics/customeraggregatespend/

• Format: Parquet

• Mode: Overwrite (full refresh)

• Register/update table in Glue Data Catalog:

• Database: gen_ai_poc_databrickscoe

• Table: customeraggregatespend

• Validation Rules:

• PricePerUnit must be numeric (castable to Decimal)

• Qty must be numeric (castable to Integer or Decimal)

• CustId must exist in both customer and order datasets for join

• Name is not assumed to be unique (multiple customers may have same name)

• TotalAmount must be non-negative (business rule assumption)

• Date must be valid date format

• Error Handling and Logging:

• Log INFO: Start of customer aggregate spend calculation

• Log INFO: Customer records read: {count}

• Log INFO: Order records read: {count}

• Log INFO: Joined records (customer + order): {count}

• Log WARNING if join produces zero records: "No matching records between customer and order datasets"

• Log WARNING: Records excluded due to non-numeric PricePerUnit: {count}

• Log WARNING: Records excluded due to non-numeric Qty: {count}

• Log INFO: Records after numeric validation: {count}

• Log INFO: Aggregated records (unique Name + Date combinations): {count}

• Log ERROR and FAIL job if aggregation fails due to data type issues

• Log ERROR and FAIL job if S3 write fails with path and error details

• Log ERROR and FAIL job if catalog registration fails

• Log INFO: Customer aggregate spend table created successfully

• Log INFO: Total records in customeraggregatespend: {count}

• Runtime Parameters:

• aggregate_target_path: s3://adif-sdlc/analytics/customeraggregatespend/ (default, overridable)

• aggregate_table_name: customeraggregatespend (default, overridable)

• Operational Notes:

• Full refresh approach (overwrite mode) ensures data consistency

• For incremental aggregation, consider partitioning by Date and using append mode

• Parquet format provides efficient storage and query performance for analytics

• Table will be queryable via Amazon Athena after registration

• Consider adding partition by Date if dataset grows large

• Ensure Glue job has s3:PutObject permission on target path

• Ensure Glue job has glue:CreateTable and glue:UpdateTable permissions

•

3. Runtime Strategy Notes

Authoring Language

• Primary Language: PySpark (Python with Spark APIs)

• Glue Context: AWS Glue PySpark with GlueContext for catalog integration

• Spark Version: 3.1 or higher (Glue 3.0+)

• Python Version: 3.7 or higher

Execution Strategy

• Job Type: AWS Glue ETL Job (Spark)

• Execution Model: Batch processing

• Trigger Mechanism: On-demand or scheduled (not defined in FRD)

• Parallelism: Leverage Spark's distributed processing across Glue DPUs

• Optimization: Use Glue DynamicFrame for catalog integration, convert to DataFrame for complex transformations

Glue Job Type Expectations

• Glue Version: 3.0 or higher (required for Hudi support)

• Worker Type: G.1X or G.2X (standard workers)

• Number of Workers: Minimum 2 (1 driver + 1 executor), scale based on data volume

• Job Timeout: 2880 minutes (48 hours) default, adjust based on data volume

• Max Retries: 0 (fail fast for data quality issues)

• Job Bookmark: Disabled (full refresh processing model)

• Continuous Logging: Enabled for real-time monitoring

• Spark UI: Enabled for debugging and performance analysis

•

4. Data Design Details

Source Paths / Source Systems

ource Identifier	Source Type	Source Path	Format	Schema Reference
---	---	---	---	---
ustomer CSV	S3 CSV Files	s3://adif-sdlc/sdlc_wizard/customerdata/	CSV	TR-INGEST-001
rder CSV	S3 CSV Files	s3://adif-sdlc/sdlc_wizard/orderdata/	CSV	TR-INGEST-002
lc_wizard_customer	Glue Catalog Table	gen_ai_poc_databrickscoe.sdlc_wizard_customer	Parquet	TR-CATALOG-00
lc_wizard_order	Glue Catalog Table	gen_ai_poc_databrickscoe.sdlc_wizard_order	Parquet	TR-CATALOG-00
dersummary (existing)	Hudi Table	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Hudi COPY_ON_WRITE	TR-SCD2-001

• Note: All sources listed above are the ORIGINAL sources as specified in the FRD. No intermediate staging tables or derived sources are introduced.

Target Paths / Target Tables

fier	Target Type	Target Path	Format	Write Mode	Schema
---	---	---	---	---	---
customer	Glue Catalog Table	s3://adif-sdlc/catalog/sdlc_wizard/customer/	Parquet	Overwrite	TR-CA
order	Glue Catalog Table	s3://adif-sdlc/catalog/sdlc_wizard/order/	Parquet	Overwrite	TR-CA
y	Hudi Table + Glue Catalog	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Hudi COPY_ON_WRITE	Upsert	TR-SC
regatespend	Glue Catalog Table	s3://adif-sdlc/analytics/customeraggregatespend/	Parquet	Overwrite	TR-AG

Catalog Registration Expectations

	Table Name	Database	Registration Method	Update Frequency	
	---	---	---	---	
	sdlc_wizard_customer	gen_ai_poc_databrickscoe	Glue write_dynamic_frame.from_catalog()	Every job run (overwrite)	
	sdlc_wizard_order	gen_ai_poc_databrickscoe	Glue write_dynamic_frame.from_catalog()	Every job run (overwrite)	
	ordersummary	gen_ai_poc_databrickscoe	Hudi automatic sync	Every job run (incremental)	
	customeraggregatespend	gen_ai_poc_databrickscoe	Glue write_dynamic_frame.from_catalog()	Every job run (overwrite)	

Partitioning Expectations

	Table Name	Partitioning Strategy	Partition Columns	Rationale	
	---	---	---	---	
	sdlc_wizard_customer	None	N/A	Small dataset, no partitioning needed	
	sdlc_wizard_order	None	N/A	Initial implementation, consider Date partitioning if dataset grows	
	ordersummary	None	N/A	Hudi manages internal partitioning, no explicit partition columns	
	customeraggregatespend	None	N/A	Consider Date partitioning for incremental processing in future	

File Formats / Table Formats

	Table Name	Storage Format	Compression	Rationale	
	---	---	---	---	
	sdlc_wizard_customer	Parquet	Snappy (default)	Efficient columnar storage, good compression	
	sdlc_wizard_order	Parquet	Snappy (default)	Efficient columnar storage, good compression	
	ordersummary	Hudi (Parquet base)	Snappy (default)	SCD2 support, time-travel queries, ACID transactions	
	customeraggregatespend	Parquet	Snappy (default)	Efficient analytics queries, good compression	

Update / Overwrite / Append Behavior

	Table Name	Write Mode	Behavior	Justification	
	---	---	---	---	
	sdlc_wizard_customer	Overwrite	Full refresh on each run	Catalog tables reflect current cleaned state	
	sdlc_wizard_order	Overwrite	Full refresh on each run	Catalog tables reflect current cleaned state	
	ordersummary	Upsert (Hudi)	Incremental SCD2 updates	Maintain historical versions, efficient updates	
	customeraggregatespend	Overwrite	Full refresh on each run	Aggregation recalculated from source on each run	

• Note: No staging tables, intermediate layers, or derived sources are introduced. All processing flows directly from original sources (CSV files or catalog tables) to final targets.

•

5. Processing Details

Data Quality Rules

	Rule ID	Rule Description	Applied To	Implementation	
	---	---	---	---	
	DQ-001	Remove string "Null" values	Customer, Order	Filter: col(column) != "Null" for all columns	
	DQ-002	Remove actual NULL values	Customer, Order	Filter: col(column).isNotNull() for all columns	
	DQ-003	Remove duplicate records	Customer, Order	DataFrame.dropDuplicates() on all columns	
	DQ-004	Validate numeric types	Order (PricePerUnit, Qty)	Cast to Decimal/Integer, exclude records where cast fails	
	DQ-005	Validate date format	Order (Date)	Parse using Spark date functions, exclude unparseable dates	

Deduplication Rules

	Dataset	Deduplication Key	Method	Timing	
	---	---	---	---	
	Customer	All columns	Exact match across all columns	After null removal, before catalog write	
	Order	All columns	Exact match across all columns	After null removal, before catalog write	

• Note: Deduplication considers exact match on all columns. No specific business key is used for deduplication at this stage.

Join Rules

	Join ID	Left Dataset	Right Dataset	Join Key	Join Type	Null Handling	
	---	---	---	---	---	---	
	JOIN-001	Order	Customer	CustId	Inner	Exclude records with null CustId	
	JOIN-002	Order	Customer	CustId	Inner	Exclude records with null CustId	

• Join Details:

• JOIN-001 (for SCD2): order_df.join(customer_df, on="CustId", how="inner")

• Purpose: Enrich order data with customer attributes for ordersummary

• Null Handling: Inner join automatically excludes null CustId values

• JOIN-002 (for Aggregation): order_df.join(customer_df, on="CustId", how="inner")

• Purpose: Enrich order data with customer Name for aggregation

• Null Handling: Inner join automatically excludes null CustId values

Aggregation Rules

	Aggregation ID	Aggregation Description	Group By Columns	Aggregate Columns	Aggregate Function	
	---	---	---	---	---	
	AGG-001	Customer total spending	Name, Date	Amount (PricePerUnit × Qty)	SUM	

• Aggregation Details:

• AGG-001:

• Calculation: TotalAmount = SUM(PricePerUnit × Qty)

• Grouping: GROUP BY Name, Date

• Output: One row per unique (Name, Date) combination

• Note: Name is not assumed to be unique across customers

SCD / History Tracking Rules

Type	Table Name	Business Key	Change Detection Attributes	Tracking Columns	Implementation
---	---	---	---	---	---
Type 2	ordersummary	OrderId	Name, EmailId, Region	IsActive, StartDate, EndDate, OpTs	Hudi UPSERT with version manag

• SCD Type 2 Implementation Details:

1. Business Key: OrderId

• Uniquely identifies each order record

• Used as Hudi record key for upsert operations

2. Change Detection Attributes: Name, EmailId, Region

• Customer attributes that trigger new versions when changed

• Compared between existing active records and new customer data

3. Tracking Columns:

• IsActive (Boolean): True for current version, False for historical versions

• StartDate (Date): Effective start date of this version (current_date() on insert/update)

• EndDate (Date): Effective end date of this version (NULL for active, current_date() when closed)

• OpTs (Timestamp): Operation timestamp (current_timestamp() on insert/update)

4. Version Management Logic:

• New Record: IsActive=True, StartDate=current_date(), EndDate=NULL, OpTs=current_timestamp()

• Change Detected:

• Close old version: IsActive=False, EndDate=current_date()

• Create new version: IsActive=True, StartDate=current_date(), EndDate=NULL, OpTs=current_timestamp()

• No Change: No action, existing active record remains unchanged

5. Hudi Configuration:

• Record Key: OrderId

• Precombine Key: OpTs (latest timestamp wins in case of conflicts)

• Operation: UPSERT

• Table Type: COPY_ON_WRITE

Null Handling

	Scenario	Handling Strategy	Implementation	
	---	---	---	
	String "Null" in source CSV	Remove record	Filter: col(column) != "Null"	
	Actual NULL in source CSV	Remove record	Filter: col(column).isNotNull()	
	NULL in join key (CustId)	Exclude from join	Inner join automatically excludes	
	NULL in numeric fields (PricePerUnit, Qty)	Exclude from aggregation	Cast validation, exclude failed casts	
	NULL in EndDate (SCD2)	Valid for active records	Explicitly set to NULL for active records	

Type Casting Expectations

	Column Name	Source Type	Target Type	Casting Method	Error Handling	
	---	---	---	---	---	
	PricePerUnit	String (CSV)	Decimal	cast("decimal(10,2)")	Exclude records where cast fails	
	Qty	String (CSV)	Integer	cast("int")	Exclude records where cast fails	

	Date	String (CSV)	Date	to_date(col("Date"))	Exclude records where parsing fails	
	IsActive	N/A	Boolean	lit(True) or lit(False)	N/A (derived column)	
	StartDate	N/A	Date	current_date()	N/A (derived column)	
	EndDate	N/A	Date	lit(None).cast("date")	N/A (derived column)	
	OpTs	N/A	Timestamp	current_timestamp()	N/A (derived column)	

• Type Casting Notes:

• Decimal precision and scale not specified in FRD; using decimal(10,2) as reasonable default for currency

• Date format not specified in FRD; using Spark's default date parser (handles common formats)

• Failed casts should be logged with record details for data quality monitoring

•

6. Traceability Matrix

FRD to TRD Requirement Mapping

irement ID	FRD Title	TRD Requirement ID(s)	TRD Title(s)
-------	----------	----------	----------
ST-001	Load Customer CSV Files from S3	TR-INGEST-001	Implement Customer Data Ingestion
ST-002	Load Order CSV Files from S3	TR-INGEST-002	Implement Order Data Ingestion
1	Remove Null Values and Duplicates	TR-DQ-001	Implement Data Quality Processing
LOG-001	Create Glue Catalog Tables for Customer and Order	TR-CATALOG-001	Implement Glue Catalog Registration
001	Create and Maintain Order Summary with Historical Tracking	TR-SCD2-001	Implement SCD Type 2 for Order Summa
001	Trigger SCD2 Updates Based on Customer Changes	TR-INCR-001	Implement Incremental Customer Update
001	Create Customer Spending Aggregation	TR-AGG-001	Implement Customer Aggregate Spend Ca

Original Source to Technical Source Mapping

	FRD Source Path/Location	TRD Source Identifier	TRD Source Path/Location
	----------	----------	----------
	s3://adif-sdlc/sdlc_wizard/customerdata/	Customer CSV	s3://adif-sdlc/sdlc_wizard/customerdata/
	s3://adif-sdlc/sdlc_wizard/orderdata/	Order CSV	s3://adif-sdlc/sdlc_wizard/orderdata/
atalog)	gen_ai_poc_databrickscoe.sdlc_wizard_customer	sdlc_wizard_customer (catalog)	gen_ai_poc_databrickscoe.sdlc_wizard_customer
g)	gen_ai_poc_databrickscoe.sdlc_wizard_order	sdlc_wizard_order (catalog)	gen_ai_poc_databrickscoe.sdlc_wizard_order
	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	ordersummary (Hudi)	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Source Fidelity Verification:

• All sources in TRD exactly match sources specified in FRD

• No intermediate staging tables introduced

• No alternative source paths created

• No derived sources substituted for original sources

• Source-to-target lineage is direct and traceable

•

7. Additional Notes

Schema Detail Availability

• Not enough information available in the FRD to derive complete source/target schema details for:

1. Customer CSV source - Column data types, lengths, precision, scale, and nullability not specified

2. Order CSV source - Column data types, lengths, precision, scale, and nullability not specified

3. sdlc_wizard_customer catalog table - Inherits schema from CSV source (incomplete)

4. sdlc_wizard_order catalog table - Inherits schema from CSV source (incomplete)

5. ordersummary Hudi table - Partial schema available (SCD2 tracking columns defined), but source column types not fully specified

6. customeraggregatespend table - Partial schema available (TotalAmount defined as Decimal), but source column types not fully specified

• Recommendation: Conduct schema discovery exercise on sample CSV files to determine:

• Exact data types for all columns

• Appropriate precision and scale for numeric columns

• Maximum lengths for string columns

• Nullability constraints

• Any additional validation rules or constraints

Implementation Assumptions

The following assumptions have been made to enable implementation in the absence of complete specifications:

1. PricePerUnit will be cast to Decimal(10,2) for currency representation

2. Qty will be cast to Integer type

3. Date fields will use Spark's default date parser (supports ISO 8601 and common formats)

4. CustId, OrderId, Name, EmailId, Region, ItemName will be treated as String types

5. Hudi record key (OrderId) is assumed to be unique per order

6. Customer Name is not assumed to be unique (multiple customers may share same name)

7. All CSV files in source paths will be processed (no file filtering)

8. CSV files use standard UTF-8 encoding

9. No header row variations or multi-line records in CSV files

Performance Considerations

1. Data Volume Expectations: Not specified in FRD; implementation assumes small to medium datasets

2. Partitioning Strategy: Not implemented in initial version; recommend adding Date-based partitioning if datasets exceed 1GB

3. Hudi Performance: COPY_ON_WRITE table type chosen for query performance; consider MERGE_ON_READ for write-heavy workloads

4. Glue DPU Allocation: Minimum 2 DPUs recommended; scale up based on actual data volume and processing time requirements

Future Enhancements

1. Incremental Processing: Current implementation uses full refresh for most tables; consider implementing incremental logic with watermarks

2. Change Data Capture: Implement CDC mechanism for customer updates instead of full dataset comparison

3. Data Quality Metrics: Add comprehensive data quality metrics and reporting

4. Schema Evolution: Implement schema evolution handling for Hudi tables

5. Partitioning: Add date-based partitioning for improved query performance on large datasets

6. Error Recovery: Implement checkpoint and restart logic for long-running jobs

•

• End of Technical Requirements Document